# Algoritmo Q-Learning

Fabrício Barth

Insper Instituto de Ensino e Pesquisa

Fevereiro de 2026

# Algoritmo Q-Learning

- O algoritmo Q-Learning é um algoritmo do tipo *value-based* que estimam a expectativa de retorno de uma ação $a$ sendo executada em um estado $s$ de acordo com uma política

$$q_\pi(s, a)$$

- Para que agente possa identificar uma política de controle ótima este agente precisa criar um **mapeamento** entre **estados** ($S$) e **ações** ($A$).
- Este mapeamento é aprendido através de **exemplos de transições** que o agente executou

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

- Desta forma o agente consegue identificar qual é a ação $a$ com maior retorno em um determinado estado $s$.

# Algoritmo Q-Learning

- Este mapeamento é representado por uma função $Q(S, A)$ onde $S$ são todos os estados possíveis $(s_1, s_2, \cdots)$ e onde $A$ são todas as ações possíveis $(a_1, a_2, \cdots)$

| **Q-table** | $a_1$ | $a_2$ | $a_3$ | $a_4$ |
|---|---|---|---|---|
| $s_1$ | | | | |
| $s_2$ | | | | |
| $\cdots$ | | | | |
| $s_n$ | | | | |

# Algoritmo Q-Learning

**function** *Q-Learning(env, $\gamma$, $\alpha$, episódios)*
inicializar os valores de $Q(s, a)$ arbitrariamente
**for** todos os episódios **do**
  inicializar $s$ a partir de *env*
  **repeat**
    escolher uma ação $a$ para um estado $s$
    executar a ação $a$
    observar a recompensa $r$ e o novo estado $s'$
    $Q(s, a) \leftarrow$ atualizando a partir das experiências
    $s \leftarrow s'$
  **until** $s$ ser um estado final
**end for**
**return** $Q$

Poderíamos simplesmente: $Q(s, a) \leftarrow r$. Será que funciona?

# Algoritmo Q-Learning

**function** *Q-Learning(env, $\gamma$, $\alpha$, episódios)*
inicializar os valores de $Q(s, a)$ arbitrariamente
**for** todos os episódios **do**
  inicializar $s$ a partir de *env*
  **repeat**
    escolher uma ação $a$ para um estado $s$
    executar a ação $a$
    observar a recompensa $r$ e o novo estado $s'$
    $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{A'} Q(s', A') - Q(s, a)]$
    $s \leftarrow s'$
  **until** $s$ ser um estado final
**end for**
**return** $Q$

O valor de $Q(s, a)$ não é simplesmente o valor imediato do $r$. Ele deve levar em consideração toda a trajetória (**Equação de Bellman**).

# Algoritmo Q-Learning: hiperparâmetro $\alpha$

- $\alpha$ é a taxa de aprendizado ($0 < \alpha \leq 1$), quanto maior, mais valor dá ao novo aprendizado.

# Que ação escolher?

**function** Q-Learning(env, $\alpha$, $\gamma$, episódios)
inicializar os valores de $Q(s, a)$ arbitrariamente
**for** todos os episódios **do**
  inicializar $s$ a partir de *env*
  **repeat**
    *escolher uma ação a para um estado s*
    executar a ação $a$
    observar a recompensa $r$ e o novo estado $s'$
    $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{A'} Q(s', A') - Q(s, a)]$
    $s \leftarrow s'$
  **until** $s$ ser um estado final
**end for**
**return** $Q$

# *Exploration* vs *Exploitation*

Opções para seleção de ações vistas até o momento:

- Greedy
- $\epsilon$-Greedy (Epsilon-Greedy)
- Decaying $\epsilon$-Greedy

# Greedy

A escolha de uma ação para um estado é dada pela função:

**function** escolha($s$): $a$
  **return** $\arg\max_A Q(s, A)$

- Não tem exploração.
- Sempre escolhe a ação com maior valor estimado.
- Pode ficar preso em ótimos locais.

# $\epsilon$-Greedy (Epsilon-Greedy)

A escolha de uma ação para um estado é dada pela função:

**function** escolha($s$, $\epsilon$): $a$
rv = random $(0 < rv \leq 1)$
**if** $rv < \epsilon$ **then**
  **return** uma ação $\alpha$ aleatória em $A$
**end if**
**return** $\arg\max_A Q(s, A)$

Onde $\epsilon$ tem um valor constante.

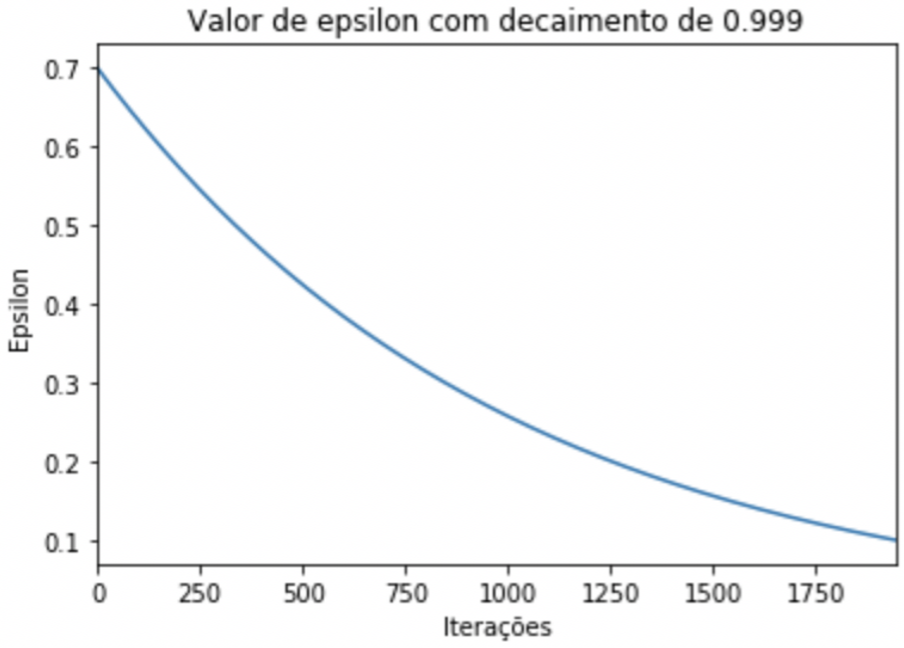
# Decaying $\epsilon$-Greedy

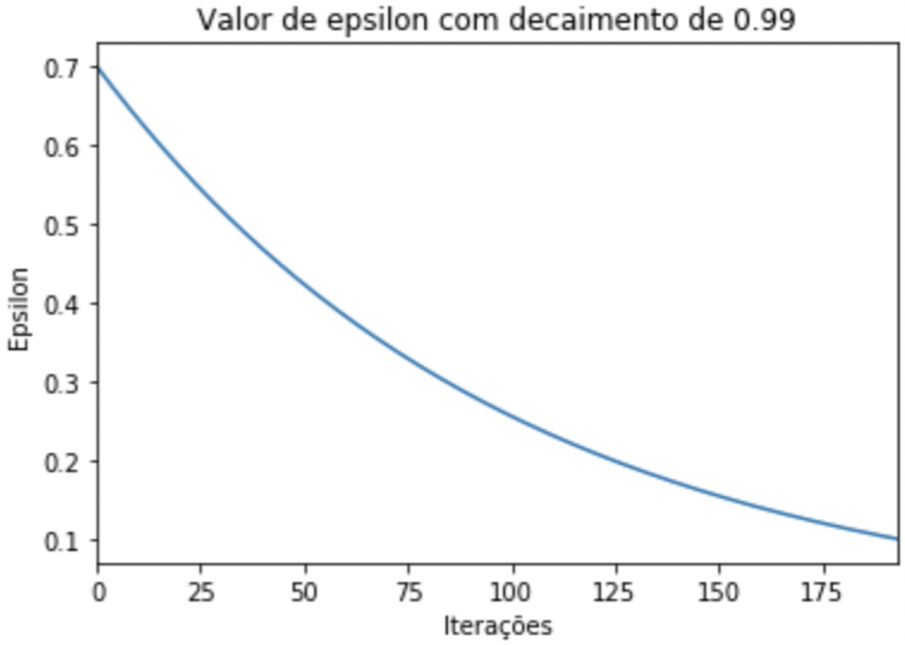
A escolha de uma ação para um estado é dada pela função:

**function** escolha($s$, $\epsilon$): $a$
rv = random $(0 < rv \leq 1)$
**if** $rv < \epsilon$ **then**
  **return** uma ação $\alpha$ aleatória em $A$
**end if**
**return** $\arg\max_A Q(s, A)$

O fator de exploração $\epsilon$ $(0 \leq \epsilon \leq 1)$ inicia com um valor alto (0.7, por exemplo) e, conforme a simulação avança, diminiu: $\epsilon \leftarrow \epsilon \times \epsilon_{dec}$, onde $\epsilon_{dec} = 0.99$

# Epsilon


Valor de epsilon com decaimento de 0.999
Epsilon
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
250
500
750
1000
1250
1500
1750
Iterações


Valor de epsilon com decaimento de 0.99
Epsilon
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
25
50
75
100
125
150
175
Iterações

# Algoritmo Q-Learning

**function** *Q-Learning(env,* $\alpha$*,* $\gamma$*,* $\epsilon$*,* $\epsilon_{min}$*,* $\epsilon_{dec}$*, episódios)*
inicializar os valores de $Q(s, a)$ arbitrariamente
**for** todos os episódios **do**
  inicializar $s$ a partir de *env*
  **repeat**
    $a \leftarrow escolha(s, \epsilon)$
    $s', r \leftarrow$ executar a ação $a$ no *env*
    $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{A'} Q(s', A') - Q(s, a)]$
    $s \leftarrow s'$
  **until** $s$ ser um estado final
  **if** $\epsilon > \epsilon_{min}$ **then** $\epsilon \leftarrow \epsilon \times \epsilon_{dec}$
**end for**
**return** Q

# Material de **consulta**


- Richard S. Sutton and Andrew G. Barto. 2018. Reinforcement Learning: An Introduction. A Bradford Book, Cambridge, MA, USA. **Capítulo 6.5**
- Watkins, C.J.C.H., Dayan, P. Q-Learning. Machine Learning 8, 279–292 (1992). Link
- Aurélien Géron. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition, 2019.